Choosing a Database by Use Case

Decision guide with real-world examples

How to Decide

The core decision axes are: **data structure**, **query patterns**, **scale**, and **consistency requirements**. No single database wins across all dimensions. The most common mistake is defaulting to PostgreSQL for everything — it handles 80% of use cases brilliantly, but purpose-built engines win decisively for time-series at IoT scale, graph traversal, or petabyte analytics.

1. E-commerce Order Management -> PostgreSQL (Relational)

Orders, customers, products, payments are highly relational with complex joins. You need ACID transactions — you cannot charge a card without creating an order atomically. Strong consistency is non-negotiable.

Why relational wins here

- A failed payment must roll back the inventory decrement — only ACID gives you that
- Complex multi-table joins: Customer -> Orders -> OrderItems -> Products -> Inventory
- Foreign key constraints enforce referential integrity across the order lifecycle

2. User Sessions / Auth Tokens -> Redis (In-Memory Key-Value)

Sessions are temporary, need sub-millisecond reads, and have a simple structure: `sessionId -> {userId, expiry, permissions}`. Redis TTL handles expiry natively. No joins needed, no long-term durability required.

Why Redis wins here

- Sub-millisecond GET/SET — a relational DB adds 5-20ms per auth check
- Native TTL: SET session:abc EX 3600 — key auto-deletes after 1 hour
- Horizontal scaling trivial; session data is never queried relationally

3. Product Catalogue / Full-Text Search -> Elasticsearch

Millions of products, users searching by keyword, filtering by facets (brand, price range, rating), ranked by relevance. Relational DBs can do this but struggle at scale. Elasticsearch is built for inverted indexes, fuzzy matching, and aggregation-heavy queries.

Typical pattern

PostgreSQL as the source of truth, synced to Elasticsearch for search. Writes go to Postgres; reads go to Elasticsearch.

Why Elasticsearch wins here

- Inverted index makes keyword search O(1) regardless of catalogue size
- Fuzzy matching handles typos: 'nkie' still finds 'Nike'
- Aggregations for faceted filters (count by brand, histogram by price) in one query

4. Social Network / Recommendations -> Neo4j (Graph DB)

'Who follows who', 'friends of friends', 'people you may know' are graph traversal problems. A relational DB can model this with a self-join table, but 5-hop traversals across millions of nodes become exponentially expensive. Neo4j navigates relationships in O(log n) regardless of graph size.

```
(Pavan)-[:FOLLOWS]->(Alice)-[:FOLLOWS]->(Bob)

MATCH (p)-[:FOLLOWS*2]->(rec) WHERE p.name = 'Pavan'
RETURN rec  -- people 2 hops away, in milliseconds
```

Why graph wins here

- Relationship traversal is O(log n) in Neo4j vs exponential cost in SQL self-joins
- Cypher query language expresses graph patterns naturally
- Built-in algorithms: PageRank, community detection, shortest path

5. IoT Sensor Data -> TimescaleDB or InfluxDB (Time-Series)

A factory floor with 10,000 sensors writing a reading every second = 864M rows/day. Queries are always time-ranged and aggregate-heavy. TimescaleDB auto-partitions by time and compresses old chunks. A generic relational DB collapses under this write volume and unindexed time scans.

TimescaleDB example

```
-- Automatic time partitioning
SELECT time_bucket('1 hour', ts) AS hour,
       AVG(temperature) AS avg_temp,
       MAX(temperature) AS peak
FROM sensor_readings
WHERE ts > NOW() - INTERVAL '7 days'
GROUP BY hour ORDER BY hour;
```

Why time-series wins here

- Chunk-based partitioning: queries only scan relevant time chunks, not the full table
- Automatic compression: old data compressed 90%+ with no query changes
- InfluxDB adds built-in downsampling policies to auto-aggregate historical data

6. Content / User-Generated Documents -> MongoDB (Document DB)

Blog posts, product descriptions, user profiles each have a different shape. A blog post has tags, embedded comments, SEO metadata; a product has specs that vary by category. Forcing this into a rigid schema means dozens of nullable columns or a messy EAV table. MongoDB stores each document as JSON with no migration needed when a new field is added.

Why document DB wins here

- Schema-less: add a new field to one document type without migrating all others
- Embed related data (comments inside a post) — one read instead of a join
- Horizontal sharding built-in for large content volumes

7. Global Leaderboard / Ranked Lists -> Redis Sorted Sets

Gaming leaderboards, trending content scores. Redis ZADD/ZRANK/ZRANGE are O(log n) and purpose-built for this. Getting rank 1-100 out of 10M users is a single command. A SQL ORDER BY score DESC LIMIT 100 on a hot table at this scale is painful.

```
# Add/update a score
ZADD leaderboard 9500 'player:pavan'

# Top 100
```

```
ZREVRANGE leaderboard 0 99 WITHSCORES

# Rank of a specific player (0-indexed)
ZREVRANK leaderboard 'player:pavan'
```

8. Financial Transactions / Ledger -> CockroachDB or Aurora PostgreSQL

Multi-region banks need ACID + geographic distribution. CockroachDB gives you Postgres-compatible SQL with serializable isolation across regions — a transfer from London to New York is consistent globally with no split-brain risk. Regular Postgres works well for single-region at smaller scale.

Key requirements met

- Serializable isolation: no dirty reads, phantom reads, or lost updates
- Multi-region replication with automatic failover
- SQL compatibility: existing Postgres tooling, ORMs, and queries work unchanged

9. Analytics / Data Warehouse -> Redshift / BigQuery / Snowflake

Running reports over 5 years of order data — aggregations across billions of rows, joining fact and dimension tables. Columnar storage reads only the columns in your SELECT, not full rows, making these queries 10-100x faster than row-oriented DBs. These are write-once, read-heavy, eventual-consistency-tolerant workloads.

Why columnar wins here

- Column pruning: SELECT revenue, region scans only 2 columns out of 50
- Compression: repeated values in a column compress 10-50x
- Massively parallel query execution across hundreds of nodes

10. Real-Time Chat / Messaging -> Cassandra (Wide-Column)

Chat history is a classic write-heavy, query-by-partition pattern: SELECT messages WHERE chat_id = X ORDER BY timestamp DESC LIMIT 50. Cassandra partitions data by chat_id, making this an O(1) lookup regardless of total message count. It handles millions of concurrent writes with linear horizontal scaling.

```
CREATE TABLE messages (
  chat_id   UUID,
  ts        TIMESTAMP,
  sender_id UUID,
  body      TEXT,
  PRIMARY KEY (chat_id, ts)
) WITH CLUSTERING ORDER BY (ts DESC);

-- This query is O(1) regardless of total rows in the table
SELECT * FROM messages WHERE chat_id = ? LIMIT 50;
```

Decision Cheat Sheet

Use Case	Database	Key Reason
Complex joins, ACID transactions	PostgreSQL / MySQL	Relational integrity
Sub-ms key lookups, caching, TTL	Redis	In-memory speed
Full-text search, faceted filters	Elasticsearch	Inverted index
Relationship traversal (social graphs)	Neo4j	Graph algorithms

Use Case	Database	Key Reason
Time-series, sensor data	TimescaleDB / InfluxDB	Time partitioning
Flexible / variable document schema	MongoDB	Schema-less JSON
Ranked lists, leaderboards	Redis Sorted Sets	O(log n) ranking
Global distribution + strong consistency	CockroachDB	Distributed ACID
Analytical queries over huge datasets	Redshift / BigQuery	Columnar storage
Write-heavy, partition-key queries	Cassandra	Linear scaling

The golden rule: *Start with your query patterns, not your data structure. Ask: 'What does my application read most often, and at what scale?' That single question eliminates most wrong choices.*